

JAVASCRIPT BASED COMMAND LINE INTERPRETER

■ BUKYA SRIDHAR

Webpages are becoming more powerful and interactive with advancements in the programming domain. Most of the current webpages have their back-ends programmed via JavaScript, which is the soul of interactive browsing. It started off in the year 1995 and is still

continuing a journey of interactivity-cum-simplicity in programming. Since there is no problem in understanding the code syntax, this program coding has seen a steep rise in today's Web world.

Simple controls in a webpage are created by using HTML (hypertext markup language) and these controls are made to work through JavaScript.



Fig. 1: Main screen of the program output



Fig. 2: Screenshot of the program when execution is successful



Here we present an interactive program (jadelclip.htm) that can be used to simulate, debug, compile and interpret simple JavaScript commands. The jadelclip.htm program can be used as a tool for debugging/editing the code and also as a command line interpreter just as MS-DOS.

The program has an editing screen area where you type the commands and codes. An embedded 'JavaScript Debugger' text is displayed on top of the screen that acts like a button. When you click the mouse on the embedded text, the JavaScript code gets compiled and the output is shown on the screen. Simple colour coding of the screen background makes it easy for you to notice the compilation and debugging process easily.

The main window of the program has a blue gradient with text input in white (refer Fig. 1). Note that the program takes only JavaScript code as input text. If you enter any other code, it will give a syntax error.

When the program is executed successfully, a window with green gradient appears (see Fig. 2), which indicates that the program has been executed without any syntax and coding errors. If there is any error in the command, a window with red gradient appears with an error message along with the line number of the code where the error occurred (see Fig. 3). If there is no code on the editor screen and the mouse cursor is placed over the embedded text, a window with orange gradient appears to remind the user that no code has been provided (refer Fig. 4).

There are two main functions that